VesSkel

Vessel Skeletonization and Graph-Based Phenotype Analysis in Retinal Fundus Images

Simon Wittmann

Supervisor: Anna Möller

9. Juni 2026

Implementation Progress

1. Euclidean Distance Transformation → Radius, Diameter
2. 39 features implemented (87 in comparison table)
3. 2D Simple Point LUT
4. Warning for unknown config settings
5. lots of new tests (9 files, → 150+ tests)
6. (not) running regression tests
7. parallelized batching (`-j` / `--jobs N`)
8. fast! shell completions (800 ms → 81 ms)
9. wavefront experiment
10. graph cleanup

Vessel Radius & Diameter via EDT

The **Euclidean Distance Transform** (EDT) computes the distance from each foreground pixel to the nearest background pixel. Sampling the EDT at skeleton positions yields the **local vessel radius** at every centerline point.

Global Statistics	Per-Segment Statistics
mean, std, min, max radius	mean, std, min, max radius
mean, std, min, max diameter	mean, std, min, max diameter

- Napari radius layer
- used for junction cleanup diameter estimation
- toggleable via `"vessel_radius": true` in config

New Features

Key additions since last update:

1. Per-segment radius & diameter stats (mean, std, min, max)
   - highest feature importance in random forest
   - plain SVM now achieves highest score
2. Per-segment tortuosity & straightness
3. Vessel area & vessel area fraction
4. HGU (hyphal growth unit = total length / endpoint count)

2D Simple Point LUT

The simple point check used a flood-fill DFS for each candidate pixel, walking the 8-neighborhood to count connected foreground components. This is called **millions of times** during thinning.

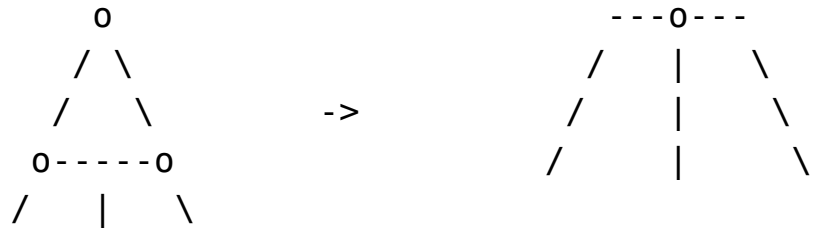
Solution: Precompute the simple point status for all 256 possible 8-neighborhood patterns.

The LUT is built at module level and shared across all thinning calls via `@njit(cache=True)`.

The same approach cannot be used for 3D (26-neighborhood = 2^{26} entries = 67 million – too large). The 3D implementation keeps the octant-based Euler check.

Junction Triangle Cleanup I

Problem: Wide vessel junctions appear as thick blobs to the thinning algorithm. Instead of producing a single junction pixel, the skeleton contains small triangle- or diamond-shaped cycles:



These artifacts inflate bifurcation counts and distort topology metrics.

Simple Algorithm:

1. Build junction graph via skan → find cycles via networkx
2. Compute perimeter of each cycle (sum of Euclidean edge lengths)
3. Estimate local vessel diameter from EDT at cycle node positions
4. Filter: collapse cycles where perimeter $<$ threshold_factor $\times$ diameter
5. Merge overlapping cycles → collapse to centroid
6. Reconnect external vessel arms

Junction Triangle Cleanup II

New Config Values:

- `"junction_cleanup": true`
- `"cleanup_threshold_factor": 5.0` (range 2.5–10)

When cleanup is enabled, the cleaned skeleton replaces the original for **every** downstream stage: saved files, summary features, napari layers.

CLI Improvements

Parallel Batch Processing

1. `vesskel run --jobs N` / `-j N`
2. `ProcessPoolExecutor` spawns workers
3. Each worker processes one image independently
4. Errors are collected, processing continues
5. full HRF now takes ≈ 23s instead of before ≈ 50s on my machine

Config quality-of-life

1. Unknown keys in config JSON produce a warning

Shell Completions

1. Generated via `argcomplete`
2. naive: ≈ 800 ms (numpy, PIL, pipeline imported eagerly)
3. After: ≈ 81 ms (heavy imports deferred after argcomplete guard)
4. Supports bash, zsh, powershell
5. Regression test enforces completion speed

Testing

File	Tests	Scope
`test_cli.py`	25	completions, input discovery, batch (seq + par)
`test_config.py`	20	serialization, validation, unknown keys
`test_features.py`	20	tortuosity, fractal dim, radii, graph
`test_pipeline.py`	25	full pipeline, all option toggles
`test_napari_layers.py`	18	layer generation, toggle combos
`test_utils.py`	12	`to_binary` behavior
`test_2d_thinning_regression.py`	45	HRF images vs saved baselines
`test_3d_thinning_regression.py`	1	brain volume vs saved baselines
`test_3d_skimage_comparison.py`	1	vesskel = skimage bit-identical

1. Regression tests now marked `@pytest.mark.slow`
2. Removed parallel test runner, not really needed anymore

Wavefront Experiment

Problem: The recheck-removal phase is sequential – removing pixel A invalidates B's precomputed simplicity check.

Key insight: Two pixels can be processed in parallel if they are **not** 8-neighbors. The 8-neighbor graph on a 2D grid has chromatic number 4.

→ Color pixels by $(c\%2, r\%2)$:

A	B	A	B	A	B	A	B
C	D	C	D	C	D	C	D
A	B	A	B	A	B	A	B
C	D	C	D	C	D	C	D

Results:

1. neglegible performance improvements (x% - 1x% faster) (second phase is not that time consuming in general)
2. different output, missing branches, not medial axis thinning anymore

Summary & Next Steps

What I did:

1. EDT-based vessel radius & diameter (global, per-segment)
2. more new features (now 39)
3. 2D Simple Point LUT → faster thinning
4. junction cleanup
5. cli improvements (parallel batching, completions, config warnings)
6. more tests
7. Wavefront experiment (4-coloring insight)

Next steps:

1. Group Feature Table into Categories
2. more features
3. 3D experiments with graph cleanup
4. more preprocessing options? simple hole filling could help on some cases in HRF
5. revisit phenotype classification